Requirements

Cacti 1.3 development series

Documentation source base: 67bc24e44000
Built: 2026-10-27

This snapshot follows the development documentation. The accompanying build manifest records each source file hash.

For Cacti 1.2, use the maintenance manual for your installed release.

Video thumbnails are represented by links; local screenshots are embedded.

Copyright (c) 2004-2026 The Cacti Group

Contents

1. Requirements 3
2. Installing Under Windows 6
3. How to Graph Your Network 12
4. Data Source Debugging 15
5. Upgrading Cacti under Windows 18
6. Hardening a Cacti Installation 20

Requirements

This manual follows the unreleased Cacti 1.3 `develop` branch. Use the 1.2.x manual for released Cacti 1.2 installations. Verify the requirements of the exact revision or release you deploy; development requirements can change.

Supported operating systems

Platform guidance checked on 26 October 2026. Use the current security-updated point release in a vendor-supported stable branch. The examples below target these maintained operating systems; vendor OS support does not certify every Cacti, PHP, database, or third-party binary combination.

Operating system	Versions used for installation guidance	Lifecycle source
RHEL	Current supported minor release of 9 or 10	Red Hat lifecycle
Rocky Linux	Current minor release of 9 or 10	Rocky releases
AlmaLinux	Current minor release of 9 or 10	AlmaLinux releases
Ubuntu Server	24.04 LTS, 26.04 LTS, or 26.10	Ubuntu lifecycle
Debian	13 stable; 12 only with applicable Debian LTS coverage	Debian lifecycle
FreeBSD	14.5-RELEASE or 15.1-RELEASE, with current patches	FreeBSD security support
Windows Server	2022 or 2025, with current updates	Server 2022, Server 2025

Check the linked lifecycle information before deployment. LTS coverage can exclude architectures or packages. Use a maintained PHP and database version that also meets this Cacti branch's requirements. A supported OS with an incompatible default PHP version is not ready for installation. Do not use a beta OS or an expired point release for these procedures.

Software Requirements

Component	Development requirement or recommendation
Web server	PHP-capable Apache, nginx, or IIS FastCGI
PHP	PHP 8.3 or later in the PHP 8 series (`^8.3` in Composer)
Database	MySQL 8.4+ or MariaDB 10.11+ recommended by the installer
RRDtool	Version 1.5 or later
SNMP	Net-SNMP command-line tools for operations using external binaries
Spine	Optional; use a version compatible with Cacti and its runtime libraries

The PHP requirement comes from the dependency manifest and runtime minimum. The database values are the branch's installer recommendations, rather than a certification of every vendor release. Check the actual server and extensions before installation.

PHP extensions

Cacti checks the CLI and web PHP environments separately. Install the extensions required by the selected revision's Composer manifest and lock file, and satisfy the installer checks. The combined requirements include:

- `ctype`, `curl`, `date`, `dom`, `filter`, `gd`, `gmp`, `hash`, `intl`, `json`, `ldap`, `mbstring`, `mysqlnd`, `openssl`, `pcre`, `PDO`, `pdo_mysql`, `Phar`, `session`, `simplexml`, `sockets`, `spl`, `sqlite3`, `standard`, `xml`, and `zlib`.
- `posix` and CLI `pcntl` on Unix.
- `com_dotnet` for the Windows installer checks.

Some modules are built into PHP. Check `php -v`, `php --ini`, and `php -m` for the CLI, then check web PHP through the installer. Enabling an extension in one PHP configuration does not enable it in the other.

The PHP SNMP extension is optional. External Net-SNMP tools are still needed for collection paths such as some SNMPv3 and bulk requests. Configure their paths and verify execution under the actual web and collector accounts.

Composer and source deployments

A source checkout or GitHub source archive needs its locked PHP dependencies. Run these commands from the Cacti source root as the deployment owner:

```
composer install --no-dev
composer check-platform-reqs --no-dev
```

Use `composer install`, which reads `composer.lock`. Do not run `composer update` during installation or upgrade. A complete release bundle can already contain `include/vendor/autoload.php`; verify its generated platform checks and the release's instructions rather than assuming every archive contains vendor files.

The current manifest requires `ext-pcntl` and `ext-posix` unconditionally. Native Windows PHP cannot supply these modules. Source dependency installation is therefore blocked on native Windows, even though the installer has Windows-specific checks. A bundled artifact must also pass its runtime platform check. See Installing Under Windows before choosing a build. Do not bypass platform checks to force installation.

Hardware Sizing

Size from the polling interval, number of data sources, retention profiles, concurrent graph requests, collector settings, and measured storage latency. Device count alone is not a reliable sizing rule. Test representative polling and graphing load before increasing the deployment.

Use storage that can sustain RRD updates and database writes within the polling window. Measure CPU, memory, I/O latency, database connections, and poll completion time. Remote collectors can distribute collection work; they also require connectivity, synchronization, and operational monitoring. Spine process and thread counts should be tuned against those measurements.

Database configuration

Enable timezone support and grant the application account `SELECT` access to `mysql.time_zone_name`. The grant does not populate that table. Import the vendor's timezone data using the procedure for your operating system.

Use `utf8mb4` and the `utf8mb4_unicode_ci` collation consistently for the Cacti database. Changing server defaults does not convert existing tables; back up and plan any conversion of an installed database separately.

Start with the database vendor's defaults and review the installer's checks for the installed engine and version. Do not copy a fixed `my.cnf` block from another server. Size global buffers and per-connection buffers together with PHP and operating-system memory. Estimate Spine's connection demand as:

```
total_connections = total_processes * (total_threads + script_servers + 1)
```

Allow additional connections for web requests, plugins, and maintenance. Inspect actual storage engines before applying memory-table sizing guidance.

Keep InnoDB doublewrite protection enabled. Disabling it removes protection against incomplete page writes; an SSD alone does not establish atomic page writes. Review the durability consequences before changing commit flushing or other crash-recovery settings.

Database variables differ by engine and release. Removed settings can prevent startup. For example, MySQL 8 no longer accepts `innodb_file_format`, `innodb_large_prefix`, or `innodb_additional_mem_pool_size`, and MariaDB 10.6 removed `innodb_buffer_pool_instances`. Validate settings against the installed server's documentation and Cacti's version-aware checks before restarting. See the MySQL reference or MariaDB reference.

Accounts and platform controls

Keep application code owned by the administrator or deployment account. Give web and collector identities the read, execute, and runtime-directory write access they need. Grant temporary installation or import access only to the required paths, and remove it afterward. Keep database credentials, CSRF secrets, private caches, RRD files, and backups inaccessible over HTTP.

Configure SELinux, AppArmor, container permissions, or IIS request filtering for the required operations. Do not remove confinement globally to make installation checks pass. Follow the platform installation guide and Hardening a Cacti Installation.

Copyright (c) 2004-2026 The Cacti Group

Installing Under Windows

Use only the supported operating systems and maintained dependencies compatible with this Cacti branch.

These instructions describe a manual installation of Cacti development (1.3) on Windows. Use the requirements for your Cacti release and supported Windows builds of PHP, your database server and RRDtool. Community Windows installers are also discussed in the Cacti forums; check their included versions and maintenance status before using them.

The examples use `C:\inetpub\wwwroot\cacti` for Cacti, `C:\php` for PHP, `C:\rrdtool` for RRDtool and `C:\spine` for optional Spine. Substitute your actual paths throughout. Use a dedicated Windows account for the poller and a separate web-server identity. Install software and configure services as an administrator; the poller does not normally need administrator privileges.

Development packaging prerequisite

The current development dependency manifest requires PHP `^8.3`, including `ext-pcntl` and `ext-posix` unconditionally. Native Windows PHP cannot supply those two extensions. A source checkout therefore cannot currently install its Composer dependencies on native Windows. A bundled artifact must also pass its generated runtime platform check. Stop if it rejects Windows and use a compatible release or a Unix installation. Do not bypass platform checks or edit generated vendor files to force an incompatible installation.

The remaining instructions apply only to an artifact that passes its platform checks. Read the PHP PCNTL platform restriction and the selected revision's dependency manifest before choosing an artifact.

Install the software

1. Choose IIS or Apache and install only the web server you intend to use on the selected address and port.
2. Extract the Cacti ZIP distribution into the chosen web directory.
3. Install PHP and its required runtime libraries using the PHP Windows installation instructions. Match the architecture and build type to your web-server integration. Use a Non-Thread-Safe build for IIS FastCGI; an Apache module requires a compatible Thread-Safe build. A path without spaces, such as `C:\php`, simplifies tool configuration.
4. Install a Windows build of RRDtool and verify its runtime dependencies. If using Spine, install a compatible Spine release and the dependencies supplied or required by that build. The PHP poller can be used without Spine.
5. Install MySQL or MariaDB. Use the server's supported Windows configuration and service-management instructions. Start with its documented defaults, then review the Cacti installer's checks for your actual server and workload.

   Do not copy a tuning block from a different database version or operating system. MySQL 8 removed `NO_AUTO_CREATE_USER`, `innodb_file_format` and `innodb_large_prefix`; these settings can prevent startup. Do not disable InnoDB doublewrite protection or reduce commit durability as an installation shortcut. Size global and per-connection buffers for the available memory and expected concurrency.
6. Install Net-SNMP command-line tools if your collection configuration needs them, for example into `C:\usr`. Cacti uses these tools for SNMPv3 requests from the web interface and PHP poller; PHP's SNMP extension alone does not provide that collection path. Verify the installed executable paths instead of assuming a particular short filename or installation directory.

Configure PHP

Copy `php.ini-production` from the PHP distribution to `C:\php\php.ini`. Set the timezone for your installation and enable the required extensions. For current PHP Windows builds, an example configuration is:

```
extension_dir = "C:\php\ext"
extension=curl
extension=pdo_mysql
extension=sqlite3
extension=gd
extension=gmp
extension=intl
extension=ldap
extension=mbstring
extension=openssl
extension=sockets
extension=com_dotnet
; Enable if using PHP's SNMP extension:
extension=snmp

date.timezone = America/Detroit
file_uploads = On
```

Use only extensions supplied for the selected PHP build and install their required DLL dependencies according to PHP's instructions. Some modules are built in. The Cacti installer checks additional modules, including SimpleXML, XML and zlib, in both the web and command-line environments. `pdo_mysql` is the database driver Cacti uses; the removed `php_mysql.dll` is not a substitute. Set memory and execution limits using the installer recommendations for your installation rather than copying values sized for another server.

Check the command-line configuration before configuring the web server:

```
C:\php\php.exe --ini
C:\php\php.exe -m
```

Set `PHPRC` to `C:\php` for the web-server and poller environments if needed to select this configuration. An absolute path to `php.exe` avoids depending on `PATH` for scheduled tasks. Ensure required DLLs are discoverable by both identities; do not copy PHP DLLs into Windows system directories.

If loading external MIBs, set `MIBDIRS` to their actual directory and allow the collecting account to update the Net-SNMP index/cache there if needed. This is separate from permission to modify executable files.

After changing PHP configuration, recycle the application pool or restart the selected web-server service. Confirm the Cacti installer reports the expected configuration and modules for both web PHP and CLI PHP.

Configure the web server

IIS with FastCGI

Follow the PHP IIS instructions and Microsoft's FastCGI configuration reference.

1. Enable the IIS CGI role service, which supplies FastCGI support.
2. Create the Cacti site/application and a dedicated application pool. Match the pool architecture to PHP. In Handler Mappings, map `*.php` through `FastCgiModule` to `C:\php\php-cgi.exe`, not `php.exe`. Use the corresponding FastCGI application entry and allow script execution for the application. PHP 8 does not provide `php8isapi.dll`; do not configure an ISAPI filter.
3. Add `index.php` to the site's Default Document list. Apply the PHP CGI settings required by the PHP IIS guide, including `cgi.force_redirect=0` for IIS. Configure authentication/impersonation deliberately so the account accessing Cacti files is known.

4. Configure an HTTPS binding and restrict access during installation. Recycle the affected application pool after configuration changes.

Apache

Follow the PHP Apache instructions for a matching Apache/PHP build. For Apache 2.4 with a compatible PHP 8 Thread-Safe module, the relevant configuration is:

```
LoadModule php_module "C:/php/php8apache2_4.dll"
<FilesMatch "\.php$">
    SetHandler application/x-httpd-php
</FilesMatch>
DirectoryIndex index.php index.html
PHPIniDir "C:/php"
```

Check Apache's configuration before restarting its service. If using FastCGI instead of the Apache module, follow that integration's PHP instructions and use the appropriate PHP build.

File permissions

Identify the effective web identity and the scheduled-task account before setting ACLs. For IIS, this may be `IIS AppPool\Cacti` for a pool named `Cacti`, or another account selected by authentication and FastCGI impersonation. See Microsoft's application-pool identity guidance. Do not assume the account is always `IUSR`.

Give the web and poller identities read access to Cacti's code and configuration, and read/execute access to PHP, RRDtool, SNMP and optional Spine programs and their dependencies. If PHP requires `cmd.exe` to launch a command, allow only the necessary read/execute access. Do not grant either identity write access to Windows system executables or to tool installation directories.

The poller needs Modify permission on `rra` and `log` and on the cache subdirectories it uses. The web identity also needs Modify permission on `log`, its configured temporary/session directories and the Cacti cache subdirectories used by features such as real-time graphs. Use the installer's permission checks to identify the required paths, including `cache/boost`, `cache/mibcache`, `cache/realtime` and `cache/spikekill`. Grant access only to the identities and paths that need it, accounting for inherited ACLs.

During installation or signed-package imports, Cacti also needs to write queries and scripts under `resource/snmp_queries`, `resource/script_server`, `resource/script_queries` and `scripts`. Grant temporary Modify access to the identity performing that operation, then remove it when finished. The installer also needs to initialize a nonempty CSRF secret, by default `include/vendor/csrf/csrf-secret.php`. Allow creation in its containing directory during setup, then retain read access to the generated secret and remove the temporary directory write permission. Do not create an empty placeholder secret.

Edit `include/config.php` as an administrator. Keep it readable by the web and poller identities but not writable by them for ordinary operation. If a specific installation operation needs to update it, grant temporary write access for that operation and remove it afterward.

Create the database

The following commands are for **Command Prompt**, not PowerShell. Use the actual path to `mysql.exe` or the equivalent MariaDB client if it is not on `PATH`. The `--password` option prompts for the database administrator's password; do not put that password on the command line.

```
mysql --user=root --password
```

Create an empty database and a dedicated application account. Replace `REPLACE_WITH_UNIQUE_DATABASE_PASSWORD` with a unique secret before executing this SQL. The example account and connection use TCP loopback consistently.

```
CREATE DATABASE cacti CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'cacti_user'@'127.0.0.1'
  IDENTIFIED BY 'REPLACE_WITH_UNIQUE_DATABASE_PASSWORD';
GRANT ALL PRIVILEGES ON cacti.* TO 'cacti_user'@'127.0.0.1';
GRANT SELECT ON mysql.time_zone_name TO 'cacti_user'@'127.0.0.1';
```

Exit the SQL client and import the schema from the same Cacti release you extracted:

```
mysql --user=root --password --default-character-set=utf8mb4 cacti < "C:\inetpub\wwwroot\cacti\cacti.sql"
```

Ensure the server's time-zone tables are populated using its documented Windows procedure. Granting access to an empty `mysql.time_zone_name` table does not populate it. Resolve the database and time-zone checks reported by the Cacti installer before completing installation.

Optional Cygwin and Spine configuration

If your Spine or RRDtool build depends on Cygwin, install the runtime packages required by that build using the official Cygwin installer. For a source build, follow the release's build instructions and select its current compiler/development dependencies. Do not replace or delete DLLs shipped with a binary distribution without checking that distribution's requirements. Avoid mixing incompatible Cygwin runtimes.

For Spine, copy `spine.conf.dist` to `spine.conf` and set the connection details to match Cacti. Replace the password placeholder with the same application secret used above, not a sample password:

```
DB_Host     127.0.0.1
DB_Database cacti
DB_User     cacti_user
DB_Pass     REPLACE_WITH_UNIQUE_DATABASE_PASSWORD
DB_Port     3306
```

Restrict access to `spine.conf` to administrators and the poller account. Verify Spine from that account before selecting it as the poller in Cacti.

Finish the Cacti installation

1. Copy `include/config.php.dist` to `include/config.php` and set its database connection. Replace the password placeholder with your application secret:

   ```
   $database_default  = 'cacti';
   $database_hostname = '127.0.0.1';
   $database_username = 'cacti_user';
   $database_password = 'REPLACE_WITH_UNIQUE_DATABASE_PASSWORD';
   $database_port     = '3306';
   $url_path          = '/cacti/';
   ```

   Adjust `$url_path` if Cacti is served at another URL path.

2. Open the Cacti URL through HTTPS. For a freshly imported development schema, sign in with application username `admin` and initial password `admin`, then choose a unique administrator password when prompted. These are separate from the database credentials. Complete the installer and resolve its dependency, database, path and permission checks before allowing general access.

3. In **Console > Configuration > Settings > Paths**, verify the executable paths against your installation. For the example layout:

Setting	Example
PHP binary	`C:/php/php.exe`
RRDtool	`C:/rrdtool/rrdtool.exe`
SNMP tools, if used	`C:/usr/bin/snmpget.exe`, `snmpwalk.exe`, `snmpbulkwalk.exe`, `snmpgetnext.exe`
Cacti log	`C:/inetpub/wwwroot/cacti/log/cacti.log`
Spine, if used	`C:/spine/spine.exe`

Use the CLI PHP executable here, not the IIS FastCGI executable. Verify the separate Spine configuration path if selecting Spine.

4. Create devices and graphs deliberately, or configure discovery and automation rules for the intended network. Installing Cacti does not establish that every discovered device exposes all measurements in its selected template.

Schedule and verify the poller

Log on as the intended poller account and test the same executable and paths that the task will use:

```
C:\php\php.exe -q C:\inetpub\wwwroot\cacti\poller.php
```

Check `log/cacti.log`, data-source polling and graph updates. RRD files are created for configured data sources; an installation without graphs need not create them. Resolve permission and configuration errors before scheduling.

Open **Task Scheduler** and choose **Create Task**:

1. Select the dedicated poller account and **Run whether user is logged on or not**, with the account rights needed for batch logon. Do not enable **Run with highest privileges** unless a documented local requirement needs it.
2. Add a daily trigger that repeats every **5 minutes** indefinitely for the default five-minute scheduler configuration. If changing Cacti's scheduler interval, adjust this trigger to match it; the scheduler interval and data source polling intervals are separate settings.
3. Add a **Start a program** action with these separate fields:
    - **Program/script:** `C:\php\php.exe`
    - **Add arguments:** `-q "C:\inetpub\wwwroot\cacti\poller.php"`
    - **Start in:** `C:\inetpub\wwwroot\cacti`
4. Under **Settings**, choose **Do not start a new instance** if the task is already running. Review any task execution limit against the configured polling cycle; do not terminate a healthy poller prematurely.

Run the task manually, then verify subsequent scheduled runs, Task Scheduler's result/history, Cacti's poller statistics and graph freshness. Recheck these under the task account after changing PHP, paths, credentials or ACLs.

References

- Cacti development (1.3) PHP dependency checks
- Cacti development (1.3) installation permission checks
- PHP Windows installation, IIS and Apache
- Microsoft FastCGI configuration and application-pool identities
- MySQL 8 removed features
- Microsoft Task Scheduler

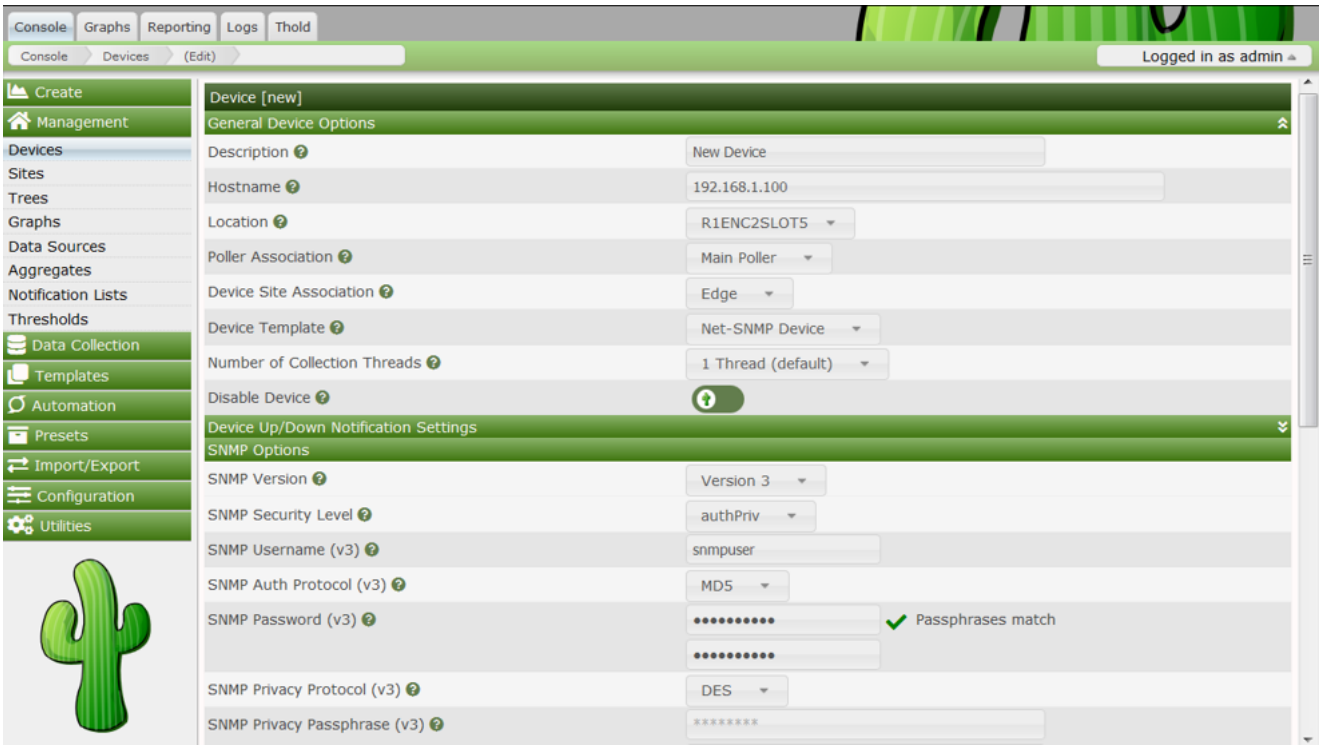
Copyright (c) 2004-2026 The Cacti Group

How to Graph Your Network

Create a device, select its measurements, and verify collection before adding its graphs to a tree. The manual workflow below is useful for a first device. Automation can perform these tasks for larger sets of devices once its discovery and graph rules are configured.

Creating a Device

Open **Console > Management > Devices** and click **Add**. Enter a description and address, choose the responsible data collector, and select a suitable Device Template. The template supplies associated graph templates and data queries; it does not restrict you to those defaults.



Device Field Definitions

- **Description** identifies the device in lists and default graph titles.
- **Hostname** is the device's address or resolvable DNS name.
- **Device Template** associates collection and graph definitions.
- **Poller Association** determines which collector reaches the device.
- **Site** groups devices by location.
- **Notes** records local information about the device.
- **Disable Device** stops collection while retaining its configuration.

Availability/Reachability Options

Choose a downed-device detection method appropriate to the device. Cacti can use SNMP, ping, both, either, or no availability test. A failed availability check can prevent measurements from being collected even when another protocol is reachable.

For ping checks, select the method, port where applicable, timeout, and retry count. ICMP permissions depend on the poller engine and operating system. Test connectivity from the collector assigned to the device.

SNMP Options

Configure the SNMP version and credentials accepted by the device:

- **SNMPv3** supports authentication and privacy. Prefer `authPriv` with mutually supported SHA and AES protocols where the device supports them.
- **SNMPv2c** supports 64-bit counters, but its community is not encrypted. Restrict access to trusted collectors and networks.
- **SNMPv1** is for legacy devices and does not support 64-bit counters.

Set the SNMP port, timeout, retries, and maximum OIDs per request. Start with values the device supports; increase request sizes only after verifying that responses are complete and collection remains reliable.

Security Options for SNMP V3

Choose the security level, username, authentication protocol and passphrase, privacy protocol and passphrase, and context or engine ID when required. The protocols offered depend on Cacti settings and the installed SNMP builds. The development defaults use SHA authentication and AES privacy. MD5 and DES are legacy algorithms and can be hidden by policy or unavailable in the build.

SNMP Retrieval Types

Cacti's PHP paths can use PHP's SNMP extension or external Net-SNMP tools. The extension supports SNMPv1, v2c, and v3; Cacti can select external tools for protocol combinations unavailable in the extension. Spine uses Net-SNMP through its C library. Verify the actual PHP, Net-SNMP, and Spine builds rather than assuming all offered algorithms are available on every platform.

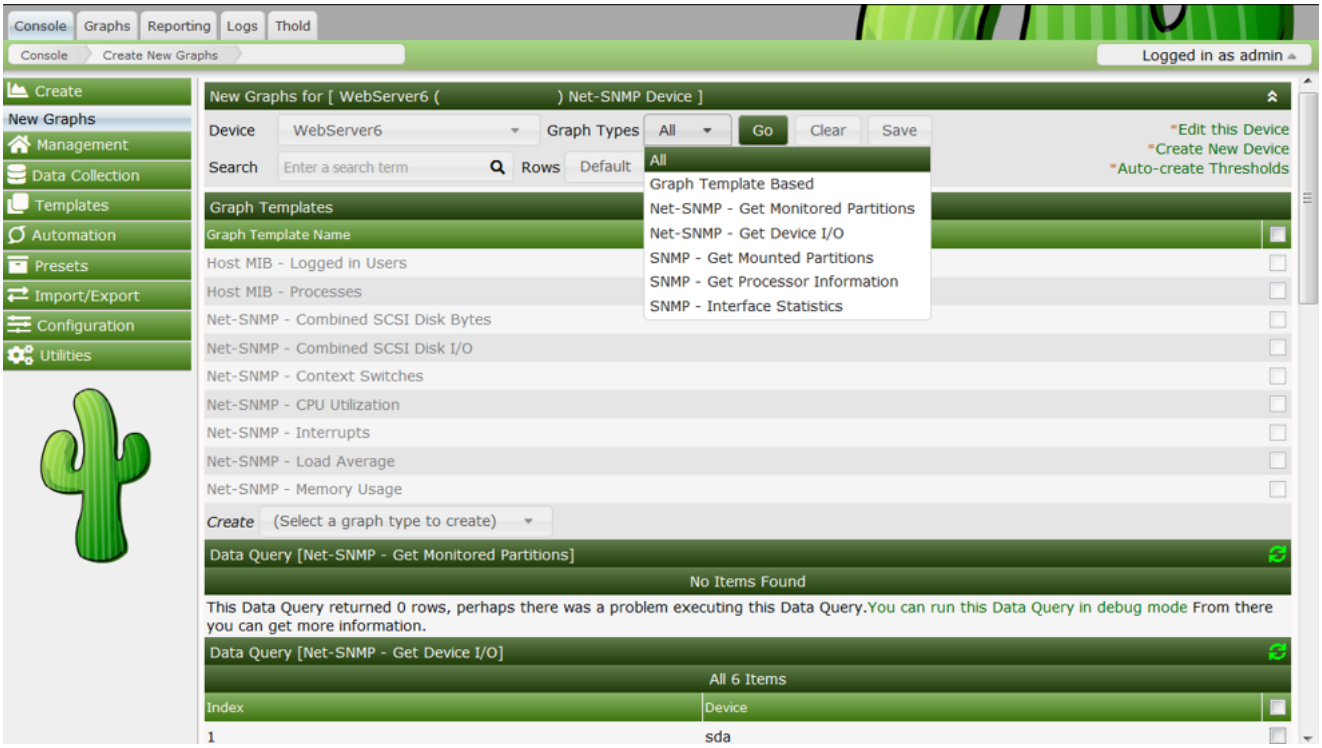
Verify device discovery

Save the device and check the SNMP information and any error shown on the edit page. Review **Associated Data Queries** and **Associated Graph Templates**. If a query has no rows, run its verbose query to check connectivity, OIDs, indexes, and credentials before creating graphs.

For interface traffic, prefer the 64-bit counter graph type when the device and selected SNMP version support it. See Data Queries for query-to-template mappings and Device Management for other availability and collection settings.

Creating the Graphs

Open **Console > Create > New Graphs** or use **Create Graphs for this Device** from the device page. Select the device, then choose the graph templates or query rows to graph. For indexed queries, choose the appropriate graph type before clicking **Create**.



Fill in any per-graph values requested by the templates and complete creation. Creating a graph defines collection; it does not immediately provide historical measurements. Allow scheduled polling to produce enough valid samples for the selected data-source type.

Verify and publish

1. Check the device's status and the Cacti log after polling.
2. Open the graph under **Console > Management > Graphs**. For blank graphs, inspect Data Source Debugging and the RRD last update.
3. Add the graph or device to a Tree, publish the tree, and grant the intended users access.
4. Confirm the result in Viewing Graphs with an account that has the intended permissions.

Copyright (c) 2004-2026 The Cacti Group

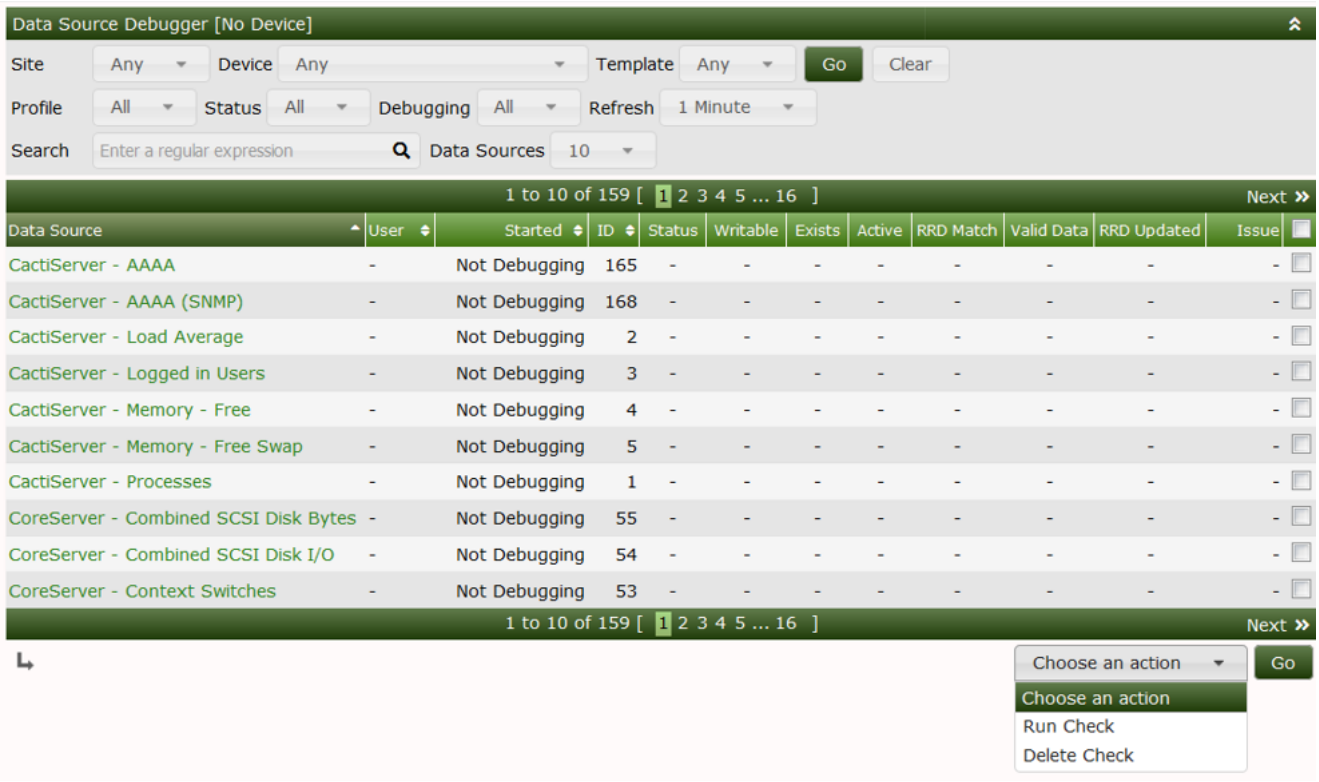
Data Source Debugging

If you are having issues with blank graphs and have done the basic checks such as permissions the Data source debug tool can help troubleshoot what information is coming into Cacti.

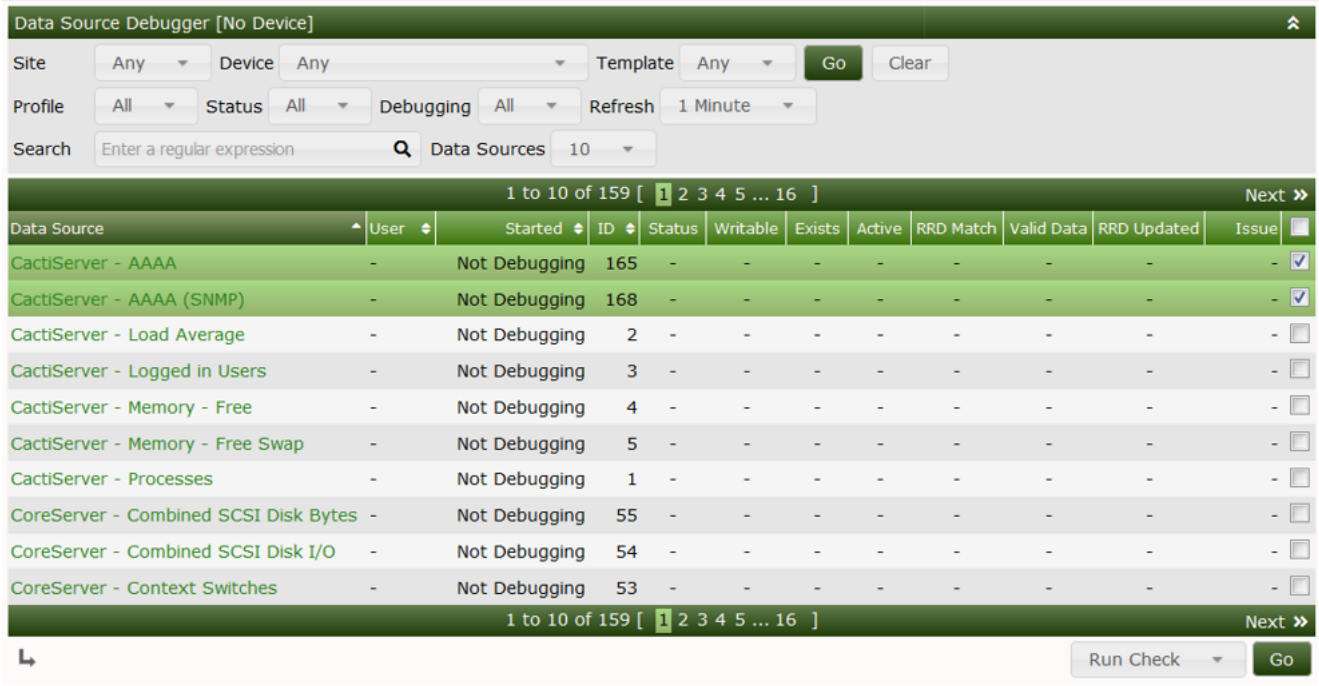
The debug tool will check the data source, graphs and permissions along incoming data and graph settings, such as step count, to ensure the definition matches the profile of the data source amongst other things.

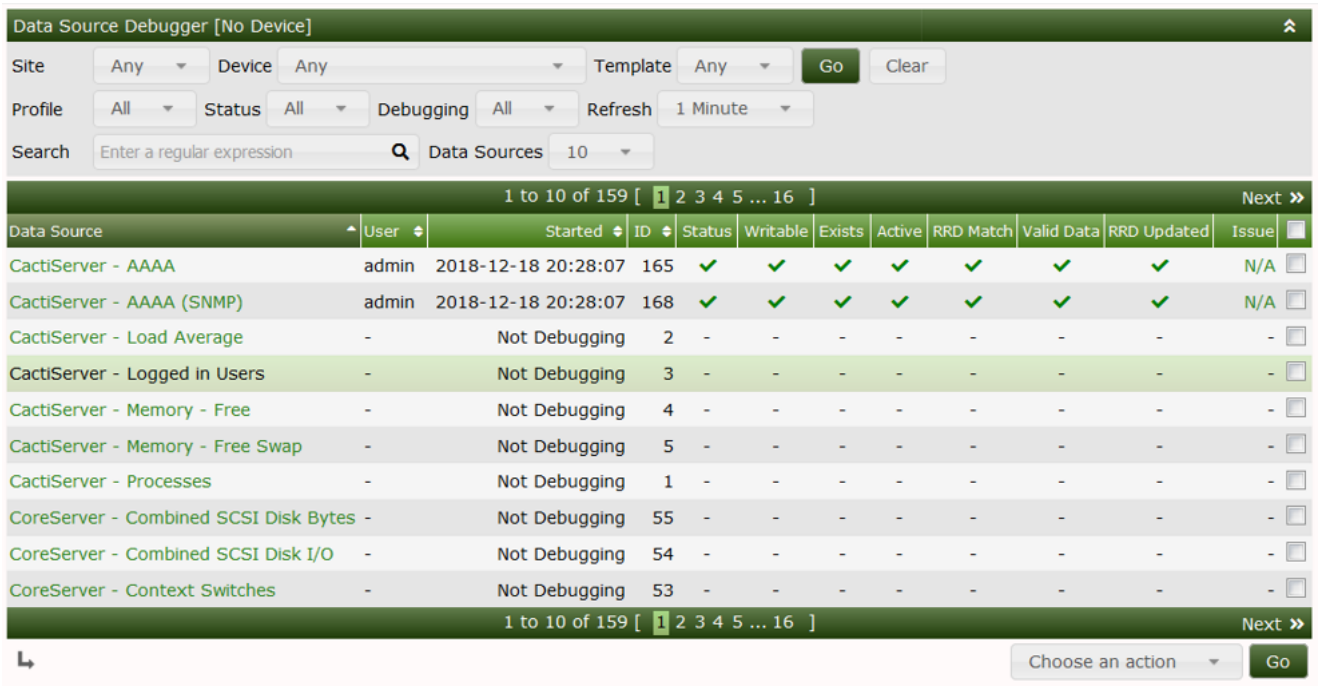
The tool allows you to run a debug on one or many data source. To request debug information, tick all applicable data sources you require information on, then select 'Run Check' in the actions drop down list. This will put all of the selected data sources into debug mode.

For a single data source, you can click the data source name which will also put it into debug mode.



You can see the status of each data source along with all of the checks run against it. Successful checks show a green checkmark, whilst failed checks shows a red X. Depending on the issue a recommendation will be provided for a failed check.





A summary of the checks as well as the properties of the data source can be displayed by clicking on the data source name. This is useful for a single page view of the data source and graph details.

Data Source Debugger		
Check	Value	Results
RRDfile Owner	apache:apache	-
Website runs as	apache	
Poller runs as	apache	
Is RRA Folder writeable by poller?	/var/www/html/cacti/rra	✔
Is RRDfile writeable by poller?	/var/www/html/cacti/rra/cactiserver_output_165.rrd	✔
Does the RRDfile Exist?	1	✔
Is the Data Source set as Active?	on	✔
Did the poller receive valid data?	output: -2.0	
Was the RRDfile updated?		Yes
First Check TimeStamp	1545182941	-
Second Check TimeStamp	1545183002	-
Were we able to convert the title?	CactiServer - AAAA	✔
Does the RRA Profile match the RRDfile structure?		✔
Issue		-

This page also shows what the last values where when last polled. This is also useful for troubleshooting as if the data comes in out of range of your Data Source template, or Graph template, this could point you into the right direction.

If the Web Server has write access to Cacti's RRA folder, if the Data Debug finds and issue that it knows how to correct. It will offer you a link to make that correction.

> **NOTE**: Devices that are down for long periods of time will report as having errors since the RRDfiles associated with them will not be updated as expected for with an available Device.

Copyright (c) 2004-2026 The Cacti Group

Upgrading Cacti under Windows

Use only the supported operating systems and maintained dependencies compatible with this Cacti branch.

These instructions cover upgrading a source/archive installation to Cacti 1.3-dev. For a managed Windows installation, also follow its package or installer upgrade procedure; do not overwrite package-managed files with an unrelated archive. Review the release notes and requirements for the target Cacti version first. An upgrade of Cacti does not automatically require upgrading Windows, PHP or the database.

The current source manifest requires `ext-pcntl` and `ext-posix`, which native Windows PHP cannot provide. Read the Windows installation prerequisite before scheduling downtime. Stop if the selected artifact rejects Windows.

Check development dependencies

Development source requires PHP 8.3 or later in the PHP 8 series, with the extensions declared by the selected revision. Install locked dependencies in the staging directory with `composer install --no-dev`, then run `composer check-platform-reqs --no-dev`. Preserve this complete dependency tree when deploying the staged application. Do not run `composer update` against a live installation. A complete release bundle can already include dependencies; verify its platform checks instead of assuming that every archive is complete.

Pause collection and take a recoverable backup

1. In `Console > Configuration > Settings > Poller`, clear **Data Collection Enabled** and save. Disable the actual Windows Scheduled Task that launches `poller.php`, or stop the service that manages your poller. Record the task or service settings so you can restore them. Use Task Scheduler to disable future starts; this alone does not finish an already running poll.

2. Wait for running pollers, Spine processes, script servers and background writers to finish. Include remote collectors and plugin jobs. Restrict web access to the upgrade administrator so configuration changes cannot race the backup. Flush pending RRD-cache writes before copying RRD files and confirm that writers have stopped. Do not start a manual poll during this maintenance window.

3. Create a protected backup directory outside the web root. From Command Prompt, use the dump utility matching your database server. Replace the executable path, account, database name and output path in this example. Use the actual database host, port and TLS options for your installation. The account needs the backup privileges required by the installed server; `--password` prompts rather than putting a password in the command.

   ```
   "C:\path\to\mysqldump.exe" --user=YOUR_BACKUP_USER --password ^
     --lock-tables --quick --routines --events --triggers ^
     --databases cacti --result-file="D:\CactiBackup\cacti-YYYYMMDD.sql"
   ```

   With all Cacti writers paused, this dump example uses table locks to cover transactional and nontransactional tables in the Cacti database. A transaction-only dump does not protect nontransactional tables. Use a new output filename and verify successful completion: a failed dump may leave a partial file. Do not proceed until the backup can be restored in a separate test environment.

4. Back up the existing Cacti tree, including hidden files, using a separate destination outside the web root. For example, Robocopy can preserve file permissions as well as contents when run with the required privileges:

   ```
   robocopy "C:\inetpub\wwwroot\cacti" "D:\CactiBackup\cacti-YYYYMMDD" /E /COPYALL /SL /XJ /R:2 /W:2
   ```

   Inspect its report; exit codes of 8 or greater indicate copy failures. Confirm that the backup includes `include/config.php`, RRDs, custom scripts, query resources, plugins and secret files, including any alternate `path_csrf_secret`. Also save configured paths outside this tree, the Spine configuration, scheduler settings, web-

server configuration, database grants and required certificates/keys. Record symbolic links and junctions and back up their external targets separately; the example copies symbolic links and excludes junctions. Preserve permissions and record the installed versions. Keep this file backup and database dump together as one recovery set.

Install and verify the upgrade

1. Extract the target Cacti release into a staging directory. Keep the existing tree and verified backup available. Deploy the release files using your existing installation layout, preserving the working `include/config.php` and local resources. Compare the new `include/config.php.dist` for changes; do not replace existing credentials, collector IDs, paths or secrets with example defaults. Check plugin and custom-script compatibility before restoring or enabling them.
2. Verify file permissions for the web server and poller accounts. Open the existing Cacti URL as an administrator and complete the database upgrade. Review installer checks and logs. Use version-appropriate database settings from Requirements, rather than applying fixed global SQL tuning commands.
3. For remote collectors, keep their schedulers paused while updating their local application files and configuration and completing the applicable upgrade and synchronization. Verify each collector's reported version, connectivity and logs; completion is not guaranteed after two poll cycles. Check the matching Spine version where it is used.
4. Re-enable **Data Collection Enabled** and restore the recorded scheduler or service only after the upgrade succeeds. Check a complete poll cycle, collector status, new RRD samples and representative graphs. If recovery is necessary, stop writers again and restore the matching application files, database, RRDs and configuration together; copying old PHP files over an upgraded database is not a rollback.

Copyright (c) 2004-2026 The Cacti Group

Hardening a Cacti Installation

This page covers OS-level hardening controls for a production Cacti host. The goal is defense in depth: even if a vulnerability in Cacti's PHP code is exploited, the attacker's reach should be limited.

Why These Controls Matter

A recurring class of vulnerability in Cacti involves writing an arbitrary file into the `resource/` or `scripts/` tree and then reaching that file over the web. The package-import advisory (GHSA-7cmj-g5qc-pj88, CVE-2024-25641) is a concrete example: a crafted package archive could write `resource/test.php` and execute it as a web request. The controls below close or limit that path at the OS and web-server layers, independent of the PHP fix.

---

1. File Permissions

Cacti writes to three paths at runtime. Everything else should be owned by root (or a non-web user) and not writable by the web process.

Path	Purpose	Recommended mode
`rra/`	RRDtool data files	`www-data:www-data 750`
`log/`	Cacti log files	`www-data:www-data 750`
`cache/`	Boost / poller cache	`www-data:www-data 750`
`resource/`	XML templates	`root:root 755` (read-only for web)
`scripts/`	Data-source scripts	`root:root 755` (read-only for web)
`lib/`, `include/`	PHP libraries	`root:root 755`

Apply once after installation:

```
# Writable paths (web user only)
chown -R www-data:www-data /var/www/html/cacti/rra \
                           /var/www/html/cacti/log \
                           /var/www/html/cacti/cache
chmod -R 750              /var/www/html/cacti/rra \
                           /var/www/html/cacti/log \
                           /var/www/html/cacti/cache

# Everything else read-only for the web user
chown -R root:root         /var/www/html/cacti
chmod -R 755               /var/www/html/cacti
# Re-apply writable exceptions after the recursive chmod
chown -R www-data:www-data /var/www/html/cacti/rra \
                           /var/www/html/cacti/log \
                           /var/www/html/cacti/cache

# Protect the database credentials in config.php.  The recursive 755 above
# leaves it world-readable; restrict it to root and the web user.
chown root:www-data /var/www/html/cacti/include/config.php
chmod 640           /var/www/html/cacti/include/config.php
```

> **Note:** The `scripts/` and `resource/` paths may be written during template import or package import by a privileged CLI process running as root, not by the web process. If your workflow requires the web process to write there, use a strict code-review policy for any import and consider the compat profile instead.

---

2. Web Server — Block Direct Access to Scripts and Resources

PHP files inside `scripts/` and `resource/` must not be reachable via HTTP. These directories hold data-source scripts and XML templates; a web request to a `.php` file written there is the direct exploitation path.

Apache

`<DirectoryMatch>` is only valid in server config or `<VirtualHost>` context, not `.htaccess`. Add this inside your Cacti `<VirtualHost>`:

```
# Deny direct HTTP access to the script and resource trees.
# A PHP file written here via import cannot be reached over the web.
<DirectoryMatch "^/var/www/html/cacti/(scripts|resource)/">
    <FilesMatch "\.php$">
        Require all denied
    </FilesMatch>
</DirectoryMatch>
```

If you only have access to `.htaccess` (e.g. shared hosting), use a `FilesMatch` block inside each directory instead:

```
# scripts/.htaccess and resource/.htaccess
<FilesMatch "\.php$">
    Require all denied
</FilesMatch>
```

Or, more broadly, deny all non-XML/non-script files from direct access:

```
<Directory "/var/www/html/cacti/scripts">
    Require all denied
</Directory>

<Directory "/var/www/html/cacti/resource">
    <FilesMatch "\.php$">
        Require all denied
    </FilesMatch>
</Directory>
```

Nginx

Add inside your `server {}` block:

```
# Block PHP execution in scripts/ and resource/
location ~ ^/cacti/(scripts|resource)/.*\.php$ {
    return 403;
}
```

---

3. PHP-FPM Pool Hardening

PHP itself warns that php-fpm pools are not a true security boundary, but they provide useful knobs. Create a dedicated pool for Cacti (`/etc/php/8.x/fpm/pool.d/cacti.conf`):

```
[cacti]
user  = www-data
group = www-data

listen = /run/php/php-cacti.sock
listen.owner = www-data
listen.group = www-data
```

```
; Drop all inherited environment variables — pass only what Cacti needs.
clear_env = yes

; Only execute .php files — blocks .phtml, .php3, .phar, etc.
security.limit_extensions = .php

; Disable dangerous functions at the pool level (not overridable by ini_set).
; Cacti's web process shells out to RRDtool, fping and helper binaries via
; exec, shell_exec, system, popen and proc_open (see lib/rrd.php, lib/ping.php,
; lib/spikekill.php), so disabling those breaks graph rendering and diagnostics.
; Only passthru and pcntl_exec are unused by the web tier and safe to disable.
; For tighter control, constrain the process with open_basedir and a hardened
; PATH rather than removing the shell-exec family Cacti depends on.
php_admin_value[disable_functions] = passthru,pcntl_exec

; Restrict file operations to the Cacti tree.
; NOTE: open_basedir disables the PHP realpath cache.  Measure the
; performance impact before enabling in production.  It is an extra
; safety net, not a full security boundary (PHP documentation).
; php_admin_value[open_basedir] = /var/www/html/cacti:/tmp

; Suppress PHP version disclosure.
php_admin_flag[expose_php] = Off
```

Reload php-fpm after changes:

```
systemctl reload php8.x-fpm   # Debian/Ubuntu
systemctl reload php-fpm       # RHEL/Rocky/Alma
```

---

4. SELinux (RHEL / AlmaLinux / Rocky Linux)

SELinux ships enforcing on RHEL-family systems. These commands apply the correct file contexts so Apache can serve Cacti while limiting what the web process can write.

File Contexts

```
CACTI=/var/www/html/cacti

# Read-only web content (Apache can read, not write)
semanage fcontext -a -t httpd_sys_content_t      "${CACTI}(/.*)?"
semanage fcontext -a -t httpd_sys_content_t      "${CACTI}/scripts(/.*)?"
semanage fcontext -a -t httpd_sys_content_t      "${CACTI}/resource(/.*)?"

# Read-write content for runtime paths only
semanage fcontext -a -t httpd_sys_rw_content_t   "${CACTI}/rra(/.*)?"
semanage fcontext -a -t httpd_sys_rw_content_t   "${CACTI}/log(/.*)?"
semanage fcontext -a -t httpd_sys_rw_content_t   "${CACTI}/cache(/.*)?"

# Apply the contexts
restorecon -Rv "${CACTI}"
```

Database Network Access

If your database server is reached over TCP (not a local Unix socket), enable the SELinux boolean that permits it:

```
setsebool -P httpd_can_network_connect_db on
```

Poller Systemd Drop-in

The Cacti spine poller runs as a separate daemon and may need to connect outward for SNMP. Do not force it into the `httpd_t` domain: that confines spine to Apache's policy rather than freeing it, and outbound access in `httpd_t` is still gated by booleans. If SNMP polling is blocked by SELinux, enable the relevant boolean instead and leave the daemon in its own domain:

```
setsebool -P httpd_can_network_connect on
```

Run `systemctl daemon-reload && systemctl restart cactid` after changes.

> **Verify:** After applying contexts, confirm Apache can still serve pages and the poller can poll. Use `audit2why` on any AVC denials in `/var/log/audit/audit.log`.

---

5. AppArmor (Debian / Ubuntu)

Ubuntu Server ships AppArmor enforcing by default. Create a profile for the Cacti php-fpm pool at `/etc/apparmor.d/cacti-fpm`:

```
#include <tunables/global>

profile cacti-fpm /usr/sbin/php-fpm* {
    #include <abstractions/base>
    #include <abstractions/php>
    #include <abstractions/mysql>

    # Cacti tree — read-only by default
    /var/www/html/cacti/           r,
    /var/www/html/cacti/**         r,

    # Allow execution of Cacti PHP files
    /var/www/html/cacti/*.php      r,
    /var/www/html/cacti/**/*.php   r,

    # Runtime write paths
    /var/www/html/cacti/rra/**     rw,
    /var/www/html/cacti/log/**     rw,
    /var/www/html/cacti/cache/**   rw,

    # Helper binaries Cacti shells out to.  AppArmor is default-deny once
    # file rules are present, so these need explicit execute (ix) or graphs
    # and polling break under enforce mode.
    /usr/bin/rrdtool               ix,
    /usr/bin/fping                 ix,
    /usr/bin/ping                  ix,
    /usr/bin/snmpget               ix,
    /usr/bin/snmpbulkwalk          ix,
    /usr/bin/php                   ix,

    # Temp files
    /tmp/**                        rw,
    /var/tmp/**                    rw,

    # Deny write to scripts/ and resource/
    deny /var/www/html/cacti/scripts/**  w,
    deny /var/www/html/cacti/resource/** w,
}
```

Load and enforce the profile:

```
apparmor_parser -r /etc/apparmor.d/cacti-fpm
aa-enforce /etc/apparmor.d/cacti-fpm
```

Attach the Profile to the php-fpm Service

Add a drop-in at `/etc/systemd/system/php8.x-fpm.service.d/apparmor.conf`:

```
[Service]
AppArmorProfile=cacti-fpm
```

Reload and restart:

```
systemctl daemon-reload
systemctl restart php8.x-fpm
```

Use complain mode (`aa-complain`) first to test without blocking, then switch to enforce once logs show no legitimate denials:

```
# Test mode
aa-complain /etc/apparmor.d/cacti-fpm

# Enforce when ready
aa-enforce  /etc/apparmor.d/cacti-fpm
```

---

6. Verification Checklist

After applying all controls, confirm:

- [ ] `curl -s http://localhost/cacti/scripts/ss_webseer.php` returns 403
- [ ] `curl -s http://localhost/cacti/resource/` returns 403 for any `.php` inside
- [ ] The Cacti web UI loads and graphs render (rra/ is writable)
- [ ] The poller runs without errors (`tail -f /var/www/html/cacti/log/cacti.log`)
- [ ] On RHEL: no AVC denials in `ausearch -m avc -ts recent`
- [ ] On Debian/Ubuntu: no AppArmor denials in `journalctl -k | grep apparmor`

See Also

- Installing Under Ubuntu/Debian
- Installing Under Enterprise Linux (LAMP)
- Standards Security

Copyright (c) 2004-2026 The Cacti Group